

Chapter 3

Page Cache 와 Advanced File I/O

5 Lecture · 영상 85 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L11 read/write 뒤에서 실제로 일어나는 일

L12 VFS 와 file_operations

L13 Page Cache

L14 Buffered I/O / Direct I/O / mmap 비교

L15 Writeback, fsync 그리고 데이터 영속성

L11

read/write 뒤에서 실제로 일어나는 일

Chapter 3. Page Cache 와 Advanced File I/O

read() 한 줄이 Disk 에 닿기까지 여섯 층 . 그리고 대부분은 Disk 까지 가지 않는다

영상 17 분

▶ 애니메이션 [C3-1 read\(\) / write\(\) Path](#)

문제 상황

buffer	read() 횟수	시간	처리량
1 B	4194304 calls	846.9 ms	4.7 MB/s
4096 B	16384 calls	10.1 ms	6343.9 MB/s
1048576 B	64 calls	3.6 ms	17934.8 MB/s

- 같은 File 인데 1,000 배 이상 차이
- 초당 17GB? 어떤 Disk 도 이렇게 빠르지 않다 → Disk 를 읽은 것이 아니다

Chapter 3 의 질문

Lecture	질문
L11	read/write 는 Kernel 안에서 어떤 길을 지나는가
L12	File, Pipe, Device, /proc 을 어떻게 같은 read() 로 읽는가
L13	read() 를 호출하면 항상 Disk 를 읽는가
L14	Buffered, Direct, mmap 은 무엇이 다른가
L15	write() 가 성공하면 Disk 에 저장된 것인가

read() 한 번이 지나는 길

- | | | |
|------------------|---------------------------------|-------------|
| ① System Call 계층 | fd → struct file | |
| ② VFS | 권한 확인 → f_op->read_iter() | |
| ③ Page Cache | Hit → 복사하고 return | ← 대부분 여기서 끝 |
| | Miss ↓ | |
| ④ File System | File 의 Block → Disk 의 Block | |
| ⑤ Block Layer | I/O 요청, Scheduling | |
| ⑥ Device Driver | Hardware 에 명령 → Process 는 Sleep | |

buf 로 들어오는 Data 는 항상 **Page Cache** 에서 복사된 것

read() / write() Path

화면 전환 → animation.html

Cache Hit → Cache Miss → write

▶ 애니메이션 [C3-1 read\(\) / write\(\) Path](#)

fd → struct file → inode

fd Table (Process 별)	struct file (open 마다)	inode (File 마다)
3	→ { f_pos=5, flags, f_op }	→ { 크기, 권한, Disk 위치, Page Cache }
4 -(dup)	→ (같은 것)	
5	→ { f_pos=0, ... }	

Offset 과 **Flag** 는 **struct file** 에 , File 의 속성과 Page Cache 는 inode 에

/proc/PID/fd, /proc/PID/fdinfo/N

Offset 은 어디에 있는가

a 에서 5 Byte 를 읽은 뒤 : a offset=5 b=dup(a) offset=5 c=open() offset=0
Child 가 a 에서 7 Byte 읽은 뒤 : a offset=12 (Parent 에서 확인)

dup, fork → struct file 공유

open 다시 → struct file 새로 생성

2>&1 이 덮어쓰지 않는 이유 / 두 Process 의 Log 에 O_APPEND 가 필요한 이유

→ Terminal 전환

Buffer 크기와 System Call

1B 와 16B: 호출 횟수 같음 → 시간도 같음 (약 $200\text{ns} \times 420 \text{ 만}$)

- 작은 I/O 의 비용 = ①~③ 을 오르내리는 고정 비용

strace -c 에 read/write 가 수백만 번 → I/O 단위를 의심

→ Terminal 전환: ./bufsize, strace -c

Short Read, 그리고 write 의 의미

read() #1: 4096 요청 → 6 Byte ← 오류가 아니다

read/write 는 항상 **Loop** 로 , EINTR 처리

write() 성공 = **Page Cache** 에 복사되었다

이 시점에	
다른 Process 가 read	새 내용이 보인다
kill -9	안전 (Data 는 Kernel 에)
전원 차단	유실

정리

read: System Call → VFS → **Page Cache** → FS → Block → Driver

- Cache Hit 면 Memory 복사로 끝
- Offset 과 Flag 는 struct file. dup/fork 는 공유 , open 은 새로
- 작은 I/O 는 System Call 비용이 지배
- write 성공 ≠ Disk 저장

다음 : L12. f_op->read_iter() 한 줄의 의미

L12

VFS 와 file_operations

"Everything is a file" 은 구호가 아니라 구조다

영상 16 분

▶ 애니메이션 [C3-2 VFS Dispatch](#)

문제 상황

```
/etc/hostname    regular file      read()= 9 : lsp2-vm.  
/proc/uptime    regular file      size=0  read()=12 : 92.95 89.53.  
/dev/urandom     character device  read()=16 : .u.)f.....4...  
pipe             pipe / FIFO       read()= 8 : via pipe
```

똑같은 `read(fd, buf, 16)`. Data 의 출처는 Disk / 즉석 생성 / Driver / Kernel Buffer

VFS: 공통 Interface 계층

Application	open read write close stat lseek mmap ioctl
VFS	공통 객체 + 공통 기능 (경로 탐색, 권한, fd, Cache) " 어떻게 " 는 모른다 → 함수 포인터로 위임
구현	ext4 xfs tmpfs procfs sysfs pipefs sockfs Driver

= 객체 지향의 Interface 와 구현 Class. C에서는 함수 포인터 구조체

struct file_operations

```
struct file_operations {
    ssize_t (*read_iter)(...);    ssize_t (*write_iter)(...);
    loff_t  (*llseek)(...);      __poll_t (*poll)(...);
    int     (*mmap)(...);        int (*fsync)(...);
    long    (*unlocked_ioctl)(...); ...
};
```

	read_iter	mmap	poll
ext4	Page Cache 경유	O	기본
pipe	Ring Buffer	X	pipe_poll
/proc/uptime	문자열 생성	X	X

`ret = file->f_op->read_iter(...);` · 빈 칸 = 지원하지 않는 동작 (ESPIPE, ENODEV)

VFS Dispatch

화면 전환 → [animation.html](#)

ext4 File → /proc/uptime → pipe → /dev/urandom

▶ 애니메이션 [C3-2 VFS Dispatch](#)

내 System 의 File System 들

/	magic=0xef53	ext2/3/4	
/tmp	magic=0x1021994	tmpfs	← 그래서 실습은 Home 에서
/proc	magic=0x9fa0	proc	
/sys/fs/cgroup	magic=0x63677270	cgroup2	

```
cat /proc/filesystems · findmnt · stat -f -c %T <path>
```

→ Terminal 전환

VFS 의 네 객체

객체	의미	단위
superblock	Mount 된 File System	Mount 마다
inode	실체 : 속성 , Data 위치 , Page Cache. 이름 없음	File 마다
dentry	이름 : (부모 , 이름) → inode	경로 요소마다
file	열린 상태 : Offset, Flag, f_op	open 마다

stat 의 출력 = 전부 inode. **File** 이름은 **inode** 에 없다

이름과 실체는 별개다

- | | | |
|-----------------------|--------------|---------|
| 1. 생성 | inode=573813 | nlink=1 |
| 2. link | inode=573813 | nlink=2 |
| 3. rename | inode=573813 | nlink=2 |
| 4. 이름을 모두 unlink | inode=573813 | nlink=0 |
| fd 로 읽기 : still alive | | |
| 5. close → 실제 삭제 | | |

unlink = File 이 아니라 **이름**을 지운다

삭제 = nlink 0 **그리고** 열고 있는 곳 0

rename = dentry 만 교체 → 즉시 , **Atomic**

→ *Terminal 전환*

실무에서 만나는 상황

"File 을 지웠는데 Disk 공간이 안 늘어난다 " → 누군가 열고 있다 . `lsdf +L1`

- 임시 File: open 직후 unlink → 어떻게 종료되든 자동 정리

Symbolic Link = 내용이 경로 문자열인 **별도의 inode**

`dcache: find /usr` 두 번째가 빠른 이유 . `drop_caches` 의 2

정리

VFS = 공통 Interface. 실제 동작은 file_operations 로 위임

`read() = file->f_op->read_iter()`

- superblock / inode / dentry / file
- 이름 (dentry) 과 실체 (inode) 의 분리 → Hard Link, Atomic rename, 열린 File 의 삭제

다음 : L13. ext4 의 read_iter 는 Disk 보다 Page Cache 를 먼저 본다

L13

Page Cache

Linux 는 남은 Memory 를 전부 " 최근에 읽고 쓴 File 내용 " 을 보관하는 데 쓴다

영상 17 분

▶ 애니메이션 [C3-3 Page Cache Simulator](#)

문제 상황

1. Cache 비운 직후	cache	0 → 65536 pages	1184.2 ms	216 MB/s
2. 바로 다시	cache	65536 → 65536 pages	46.3 ms	5529 MB/s
3. 또 다시	cache	65536 → 65536 pages	45.8 ms	5590 MB/s

read() 를 호출하면 항상 Disk 를 읽는가 ? → 아니오

→ Terminal 전환

왜 , 어디에 , 어떤 단위로

왜	DRAM 100ns / NVMe 100μs / HDD 10ms. 1,000 ~ 100,000 배
어디에	inode 마다 (address_space). Offset → Page 를 찾는 Tree
단위	4KB Page (Folio)
공유	그 File 을 쓰는 모든 Process . read() 도 mmap 도 같은 Page
크기	제한 없음 . 남는 Memory 전부 . 필요하면 즉시 반납

Page Cache Simulator

화면 전환 → [animation.html](#)

같은 Block 다시 읽기 → Working Set > Cache → 순차 읽기와 Readahead

▶ 애니메이션 [C3-3 Page Cache Simulator](#)

free 의 buff/cache 읽는 법

	total	used	free	buff/cache	available
Mem:	3916	412	310	3193	3224

free 310MB ≠ Memory 부족 . 빈 **Memory** 는 낭비

여유는 **available** 로 판단

drop_caches(1: Page Cache, 2: dentry/inode, 3: 둘 다) 는 실험용

File 의 어느 부분이 Cache 에 있는가

```
$ ./resident cache.dat /usr/bin/gcc libc.so.6
cache.dat          65536 / 65536 pages 100.0%
/usr/bin/gcc       250 /   250 pages 100.0%
libc.so.6         412 /   555 pages  74.2%
$ ./resident -e cache.dat                                # posix_fadvise(DONTNEED). root 불필요
cache.dat          0 / 65536 pages  0.0%
```

mmap + mincore() = 접근하지 않고 상주 여부만 확인 · 표준 도구 : `fincore`

→ *Terminal 전환*

Readahead

1 Byte 읽음	cache	4 pages (16 KB)
순차로 256KB 읽음	cache	80 pages (320 KB)
FADV_RANDOM 후 무작위 4KB × 64	cache	64 pages (256 KB)

- Disk I/O 는 " 얼마나 " 보다 " 몇 번 " 이 비싸다 → 가는 김에 더 가져온다
순차 접근을 감지해 Window 를 키운다 (read_ahead_kb = 128)
- 순차 읽기가 빠른 이유의 절반은 Readahead

Cache 가 가득 차면 : Reclaim

Page	회수 방법	비용
Clean File Page	그냥 버린다	거의 없음
Dirty File Page	Disk 에 쓴 뒤 버린다	Disk 쓰기
Anonymous (Heap, Stack)	Swap 에 쓴다	Disk 쓰기 + 나중에 Major Fault

Accessed bit(L07) + Active / Inactive LRU. 한 번 훑는 큰 File 이 자주 쓰는 Page 를 다 밀어내지 못하게

Application 이 줄 수 있는 Hint

<code>posix_fadvise</code>	효과
SEQUENTIAL	Readahead 2 배
RANDOM	Readahead 끄
WILLNEED	지금 비동기로 읽어 둠
DONTNEED	Cache 에서 내보냄

예 : 매일 밤 100GB Backup → 읽은 구간마다 DONTNEED → Service 의 Cache 를 밀어내지 않는다

mmap 영역에는 `madvise()`

정리

- Page Cache = inode 마다 , Page 단위 , 모든 Process 와 read/mmap 이 공유
남는 Memory 전부 . 여유는 available
- Cache Hit 인 read = Memory 복사
mincore / fcore, posix_fadvise
- Readahead, Reclaim

다음 : L14. Cache 를 거치지 않는 길과 복사를 없애는 길

L14

Buffered I/O / Direct I/O / mmap 비교

차이는 두 가지 : Page Cache 를 거치는가 , 복사가 몇 번인가

영상 17 분

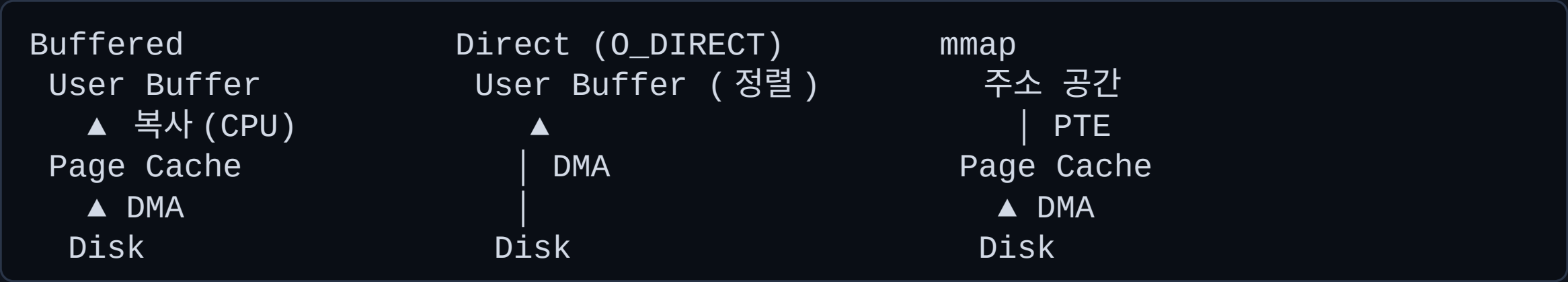
▶ 애니메이션 [C3-4 Buffered / Direct / mmap 경로 비교](#)

문제 상황

Database 는 왜 O_DIRECT 를 쓰는가 . Cache 를 안 쓰면 느려지는 것 아닌가

- mmap 은 복사가 없다는데 왜 모든 프로그램이 mmap 을 쓰지 않는가
- "O_DIRECT 는 Cache 를 안 거쳐서 빠르다 " ← 맞는 말인가

세 가지 경로



	Buffered	Direct	mmap
Page Cache	O	X	O
CPU 복사	1	0	0
접근마다 System Call	O	O	X
두 번째 접근	Hit	또 Disk	Hit

Data 는 어디를 지나는가

화면 전환 → [animation.html](#)

Buffered → Direct → mmap. 각 Scenario 의 " 다시 읽기 " 에 주목

▶ **애니메이션** [C3-4 Buffered / Direct / mmap 경로 비교](#)

측정 : Cold 와 Warm

buffered 4KB	cold 1213 ms	warm 45 ms	
buffered 1MB	cold 1187 ms	warm 31 ms	
O_DIRECT 1MB	cold 1098 ms	warm 1102 ms	← Warm 이 없다
mmap	cold 1240 ms	warm 17 ms	majflt 412 → 0

→ Terminal 전환

결과 해석

Cold 는 모두 비슷하다 . Disk 가 병목이면 위쪽 경로는 중요하지 않다

- buffered 4KB → 1MB: System Call 이 줄어든 만큼 (L11)

O_DIRECT: Warm = Cold. 매번 Disk

mmap Warm 이 가장 빠르다 . 복사 0, System Call 0

majflt: mmap 에서만 보인다 . read 의 Cache Miss 는 Page Fault 가 아니라 System Call 안의 대기

O_DIRECT 는 Kernel 의 Cache 와 Readahead 를 포기하는 것이다

O_DIRECT의 제약

```
정렬된 Buffer, 4096 B, offset 0    → 4096 Byte
어긋난 Buffer 주소                  → Invalid argument
어긋난 크기 (1000 B)                → Invalid argument
어긋난 offset (100)                 → Invalid argument
읽은 뒤 Page Cache 상주 : 0 pages
```

`posix_memalign(&buf, 4096, size)` · tmpfs 미지원 · Buffered 와 섞어 쓰지 않는다

→ *Terminal 전환*

작은 Record 의 무작위 접근

64 Byte Record 를 무작위로 2,000,000 회 (모두 Cache Hit)
 pread : 766.6 ms (383 ns / 회) System Call 2,000,000 회
 mmap : 22.0 ms (11 ns / 회) System Call 0 회

mmap 이 유리	mmap 이 불리
작게 , 무작위로 , 반복해서	큰 File 을 한 번 순차로
Memory 에 들어가는 크기	Memory 보다 훨씬 큰 File
Process 간 공유	크기가 변함 , 오류 처리 (SIGBUS), 쓰기 시점 제어

→ Terminal 전환

누가 무엇을 쓰는가, 선택 기준

	방식
cat, cp, 대부분의 Application, PostgreSQL	Buffered
MySQL InnoDB, Oracle, ScyllaDB, QEMU	O_DIRECT + 자체 Cache
LMDB, ld.so, execve	mmap

기본은 **Buffered** + 적절한 Buffer 크기 + fadvise

읽기 위주 + 무작위 + 반복 + Memory에 들어감 → **mmap**

자체 Cache + I/O 시점 직접 제어 → **O_DIRECT** (+ io_uring, Chapter 6)

측정한다 . Cold/Warm, 순차 / 무작위를 구분해서

정리

- Buffered: 복사 1 회 , Cache O. 기본
- O_DIRECT: Cache X, 복사 X, **Hit 도 Readahead 도 X**. 정렬 필요
- mmap: 복사 0, System Call 0. 작은 무작위 반복 접근에 강함
- Cold 에서는 비슷 , Warm 에서 갈린다

다음 : L15. write() 가 성공했다 . 정말 저장된 것인가

L15

Writeback, fsync 그리고 데이터 영속성

write() 의 성공은 "Kernel 이 받았다 ". "Disk 에 있다 " 는 fsync() 가 말해 준다

영상 18 분

▶ 애니메이션 [C3-5 Writeback과 fsync](#)

문제 상황

시작	Dirty	28 kB		
write() 200MB 완료 직후	Dirty	204804 kB	write :	32.1 ms (6235 MB/s)
fsync() 완료 직후	Dirty	24 kB	fsync :	1840.5 ms

- 200MB 를 32ms 에 ? → Memory 복사였다
- Dirty 가 정확히 200MB → Disk 에는 한 Byte 도 가지 않았다
- 실제 쓰기는 fsync 에서

→ Terminal 전환

Writeback: Kernel 이 알아서 쓰는 시점

계기	기본값	동작
시간	5 초마다 검사 , 30 초 지난 Dirty	Background
양 1 단계	Dirty > 10%	Background 기록 시작
양 2 단계	Dirty > 20%	write() 호출자가 Block
명시적	fsync, fdatasync, msync, sync	호출자가 대기

전원 차단 → **최대 약 30 초 분량 유실** · Crash / kill -9 → 유실 없음

왜 비동기인가

- 여러 번의 작은 쓰기를 모아 한 번에
- 같은 Page 를 100 번 고쳐도 Disk 에는 한 번
- 곧 지워질 임시 File 은 Disk 에 가지 않는다

성능을 위해 **영속성을 기본값에서 뺀 것** . 필요한 쪽이 명시적으로 요청한다

Writeback 과 fsync

화면 전환 → [animation.html](#)

write 만 → fsync → fdatasync → Background Writeback

▶ 애니메이션 [C3-5 Writeback과 fsync](#)

fsync 와 그 친척들

호출	보장
<code>write()</code>	Page Cache 에 복사
<code>fsync(fd)</code>	Data + 모든 Metadata + Disk Write Cache Flush
<code>fdatasync(fd)</code>	Data + 읽는 데 필요한 Metadata(크기) 만
<code>O_SYNC / O_DSYNC</code>	모든 write 가 fsync / fdatasync 포함
<code>sync()</code>	System 전체
<code>sync_file_range()</code>	Writeback 시작만 . 영속성 용도 아님

`close()` 는 fsync 를 하지 않는다

영속성의 가격, Group Commit

none (write 만)	1.6 ms	625,000 records/s
fdatasync 매번	2140.3 ms	467 records/s
fsync 매번	2385.7 ms	419 records/s
fdatasync 100 개마다	31.5 ms	31,746 records/s

매번 fsync = **1,000 배 이상** 느림 . 초당 fsync 횟수 = DB 의 TPS 상한

Group Commit: fsync 한 번의 비용을 여러 Record 가 나눠 가진다

PostgreSQL/MySQL WAL, Redis appendfsync everysec

→ *Terminal 전환*

File 을 안전하게 교체하기

```
fd = open("config.txt", O_WRONLY | O_TRUNC);    /* 기존 내용이 사라진다 */
write(fd, ...);                                /* 여기서 전원이 꺼지면 ? */
```

```
fd = open("config.txt.tmp", ...);    write_all(fd, ...);
fsync(fd);                           /* 새 내용이 Disk 에 */
rename("config.txt.tmp", "config.txt"); /* Atomic (L12) */
fsync(open(".", O_DIRECTORY));       /* 이름 변경도 Disk 에 */
```

어느 시점에 꺼져도 **전부 옛 내용이거나 전부 새 내용**

→ *Code + Terminal*

fsync 가 실패하면

return 값을 반드시 확인 (EIO, ENOSPC)

실패 후 다시 **fsync** → 성공 ≠ 기록됨

- Kernel 은 실패한 Page 의 Dirty 를 지우고 오류를 한 번만 보고한다
- PostgreSQL "fsyncgate" (2018)

올바른 처리 : 재시도가 아니라 그 **Data** 를 신뢰하지 않는다 (복구 또는 중단)

Chapter 3 정리

```
fd → struct file (Offset, f_op) → inode                                L11
    f_op->read_iter: ext4 / pipe / procfs / Driver (VFS)                L12
Page Cache (inode 마다, 모두가 공유)                                    L13
    read : Hit → 복사      Miss → Disk → Cache → 복사 (+Readahead)
    write: Cache 에 복사, Dirty, return
Buffered / O_DIRECT / mmap                                             L14
Dirty → Disk: Writeback(30 초, 10%, 20%) 또는 fsync                    L15
```

정리

- write = Page Cache 에 복사 + Dirty
- Writeback: 30 초 / 10% / 20%(Block)
- fsync = Data + Metadata + Disk Cache Flush. fdatasync 는 일부 Metadata 생략
- Group Commit
- tmp → fsync → rename → fsync(dir)
- fsync 실패 후 재시도 성공은 의미 없다

Worksheet 20 문제 → Chapter 4: Advanced Thread Synchronization

다음 시간 · Chapter 4

Advanced Thread Synchronization

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog